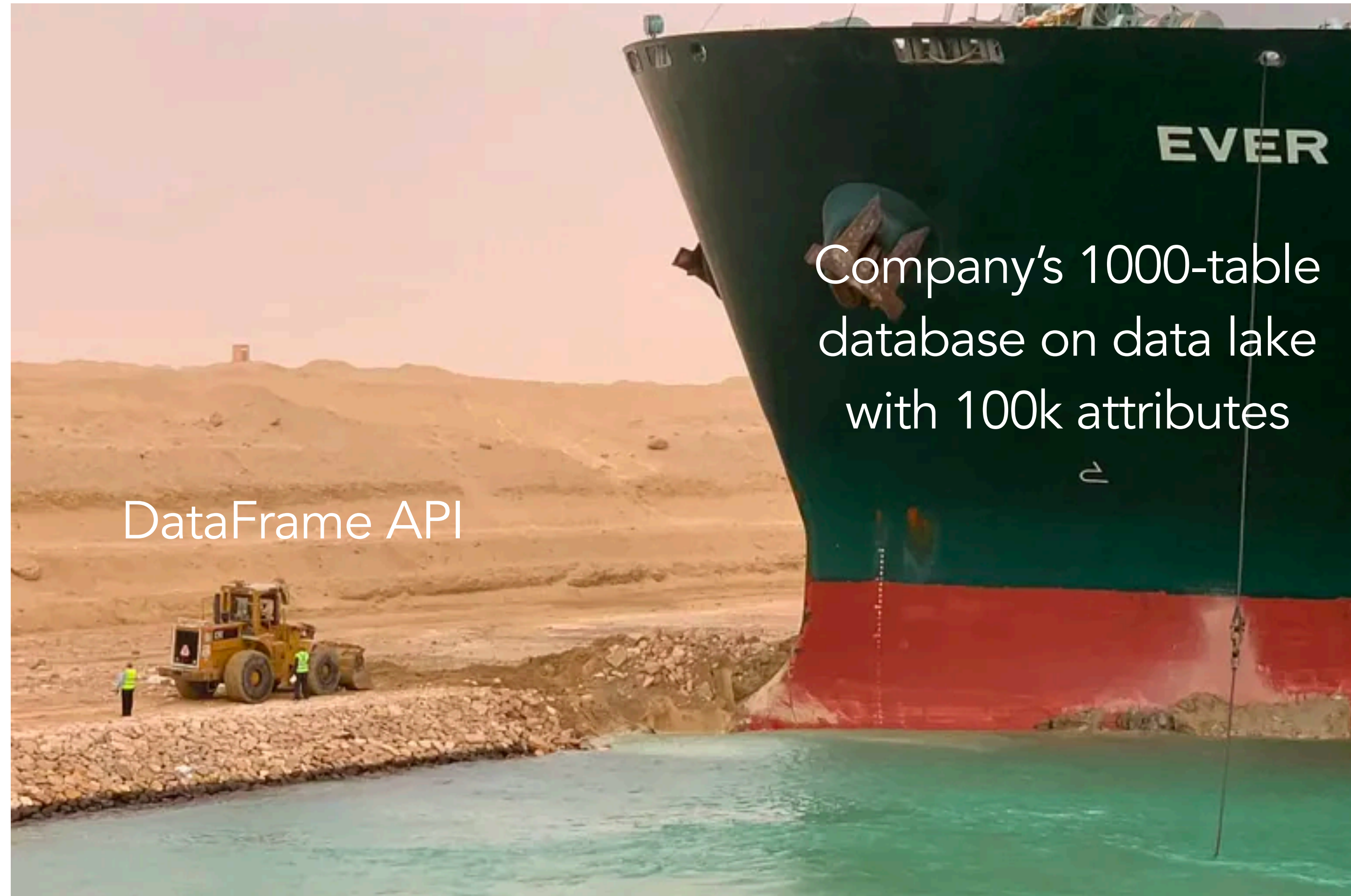


DSC 204a Scalable Data Systems

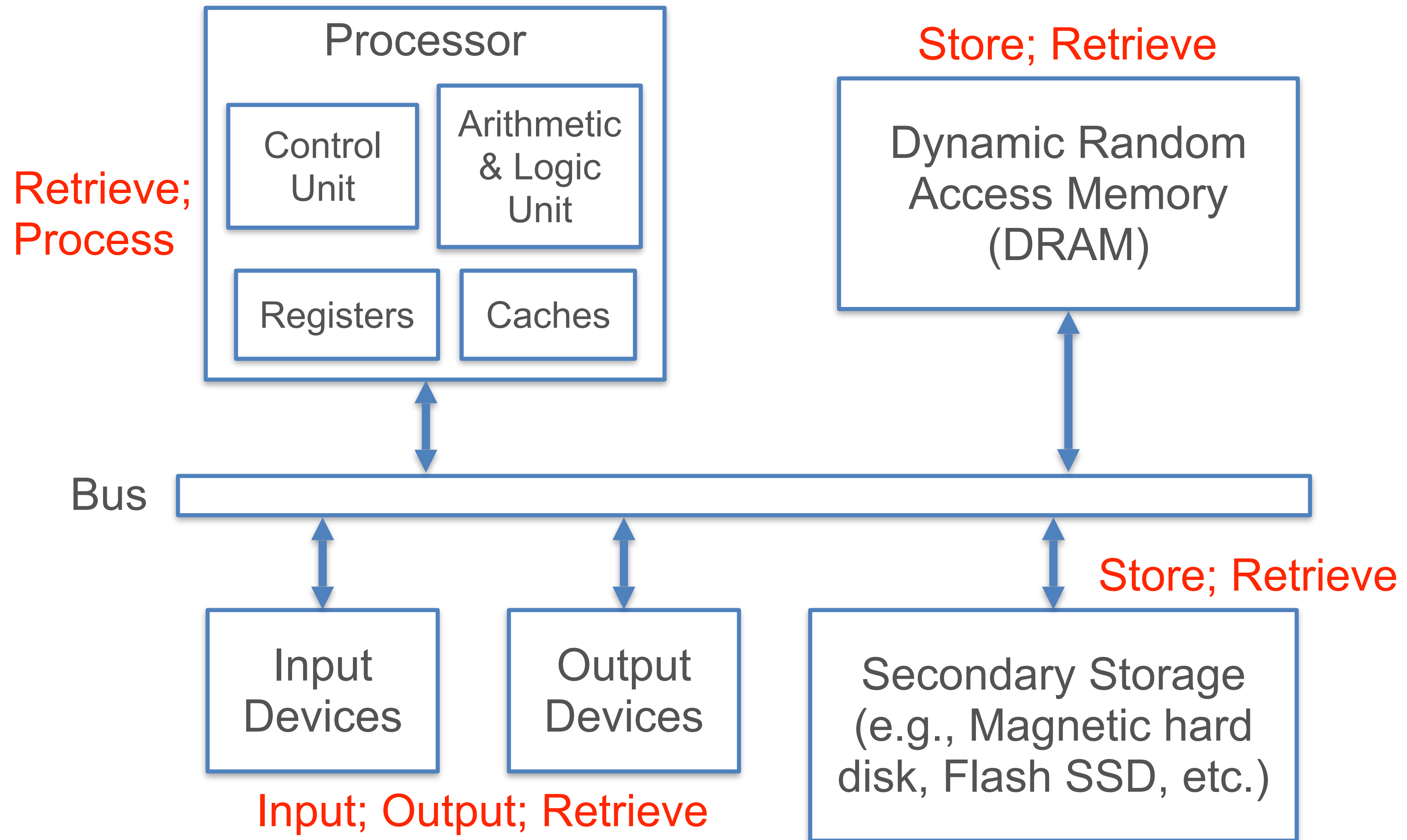
- Haojian Jin



Company's 1000-table
database on data lake
with 100k attributes

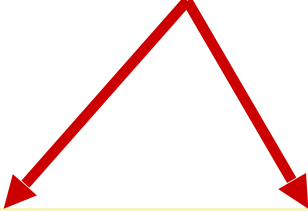
DataFrame API

Abstract Computer Parts and Data



Instruction

Register names



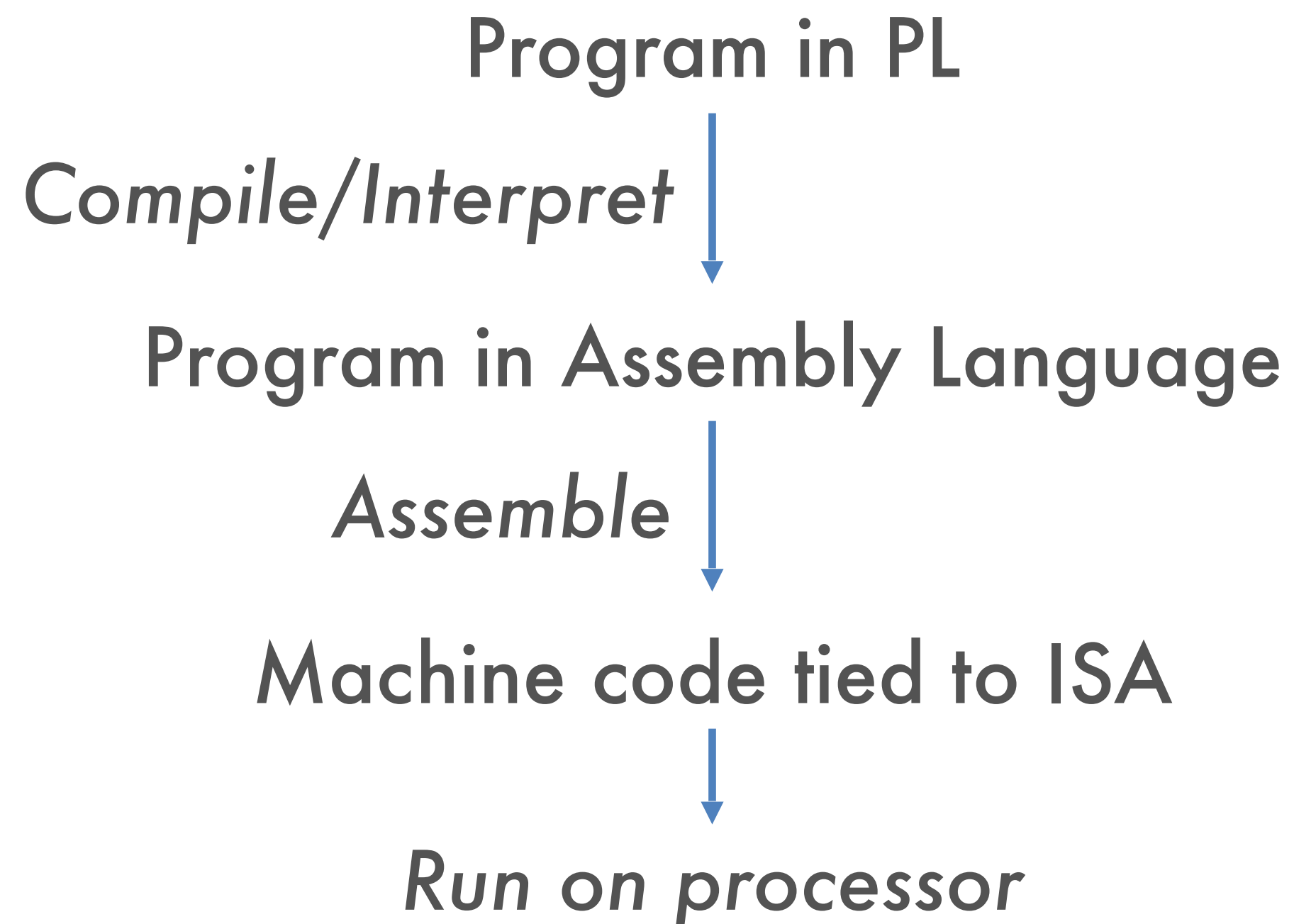
```
addq %rbx, %rax
```

is

```
rax += rbx
```


Basics of Processors

- **Processor:** Hardware to orchestrate and execute *instructions* to *manipulate data* as specified by a program
 - Examples: CPU, GPU, FPGA, TPU, embedded, etc.
- **ISA (Instruction Set Architecture):**
 - The vocabulary of commands of a processor



```
80483b4: 55          push    %ebp
80483b5: 89 e5       mov     %esp,%ebp
80483b7: 83 e4 f0    and     $0xffffffff0,%esp
80483ba: 83 ec 20    sub     $0x20,%esp
80483bd: c7 44 24 1c 00 00 00 movl    $0x0,0x1c(%esp)
80483c4: 00
80483c5: eb 11       jmp     80483d8 <main+0x24>
80483c7: c7 04 24 b0 84 04 08 movl    $0x80484b0,(%esp)
80483ce: e8 1d ff ff ff call    80482f0 <puts@plt>
80483d3: 83 44 24 1c 01 addl    $0x1,0x1c(%esp)
80483d8: 83 7c 24 1c 09 cmpl    $0x9,0x1c(%esp)
80483dd: 7e e8       jle     80483c7 <main+0x13>
80483df: b8 00 00 00 00 mov     $0x0,%eax
80483e4: c9         leave
80483e5: c3         ret
80483e6: 90         nop
80483e7: 90         nop
80483e8: 90         nop
80483e9: 90         nop
80483ea: 90         nop
```

Basics of Processors

Q: *How does a processor execute machine code?*

- Most common approach: **load-store architecture**
- **Registers:** Tiny local memory (“scratch space”) on proc. into which instructions and data are copied
- ISA specifies bit length/format of machine code commands
- ISA has several commands to manipulate register contents

Writing & Reading Memory

Write

Transfer data from memory to CPU

movq %rax, %rsp

“Store” operation

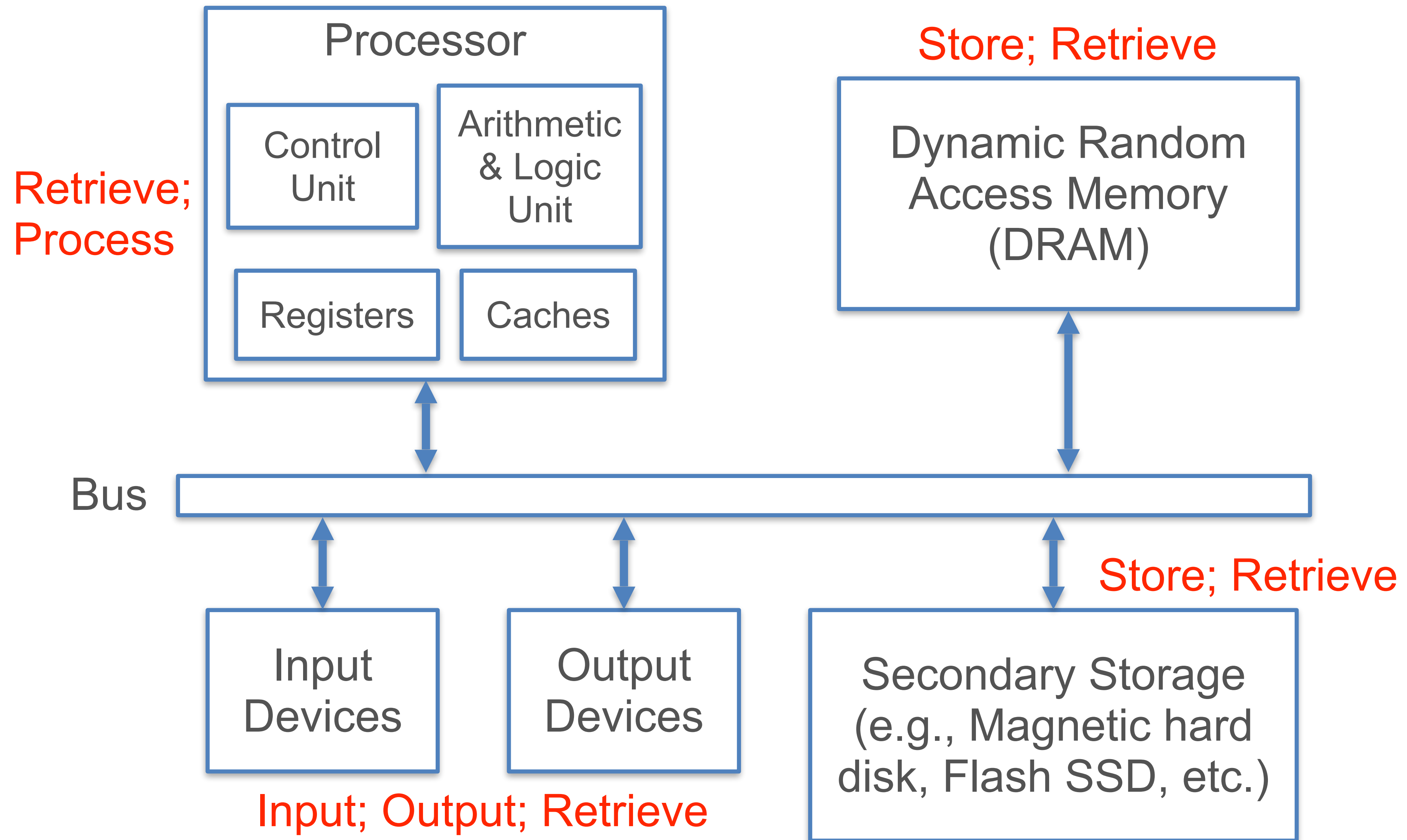
Read

Transfer data from CPU to memory

movq %rsp, %rax

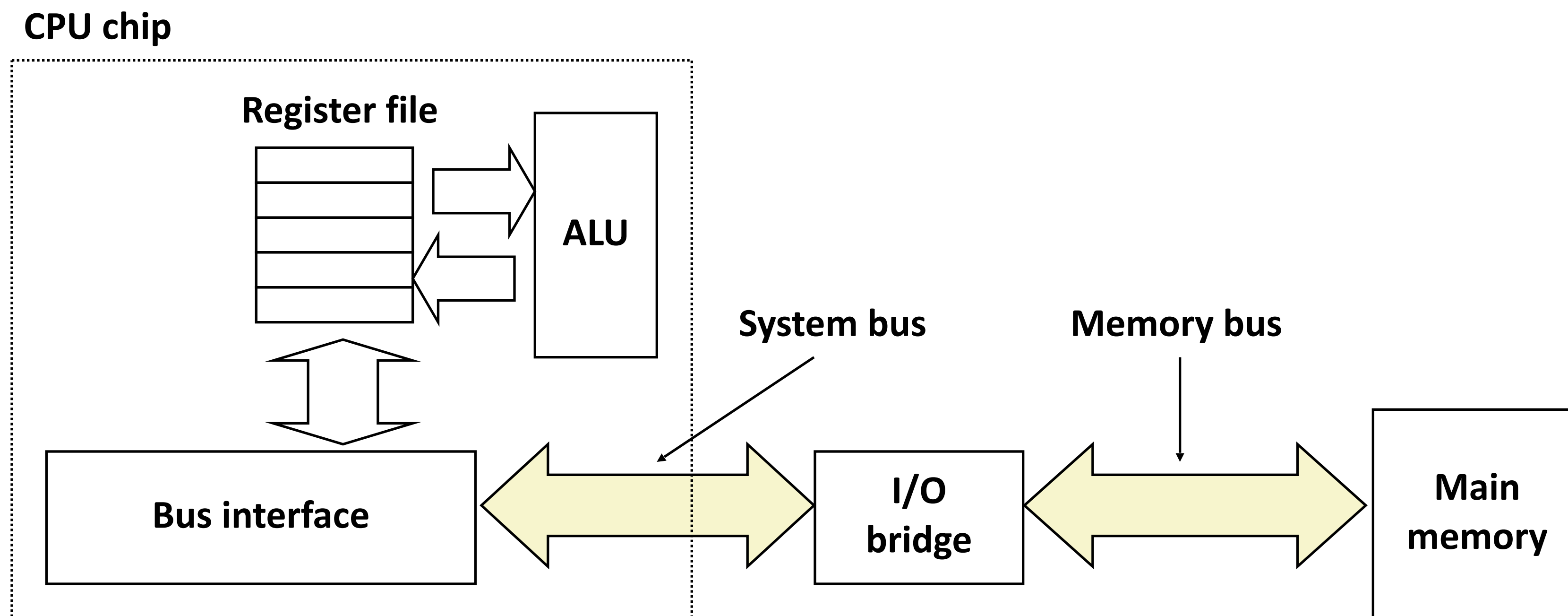
“Load” operation

Abstract Computer Parts and Data

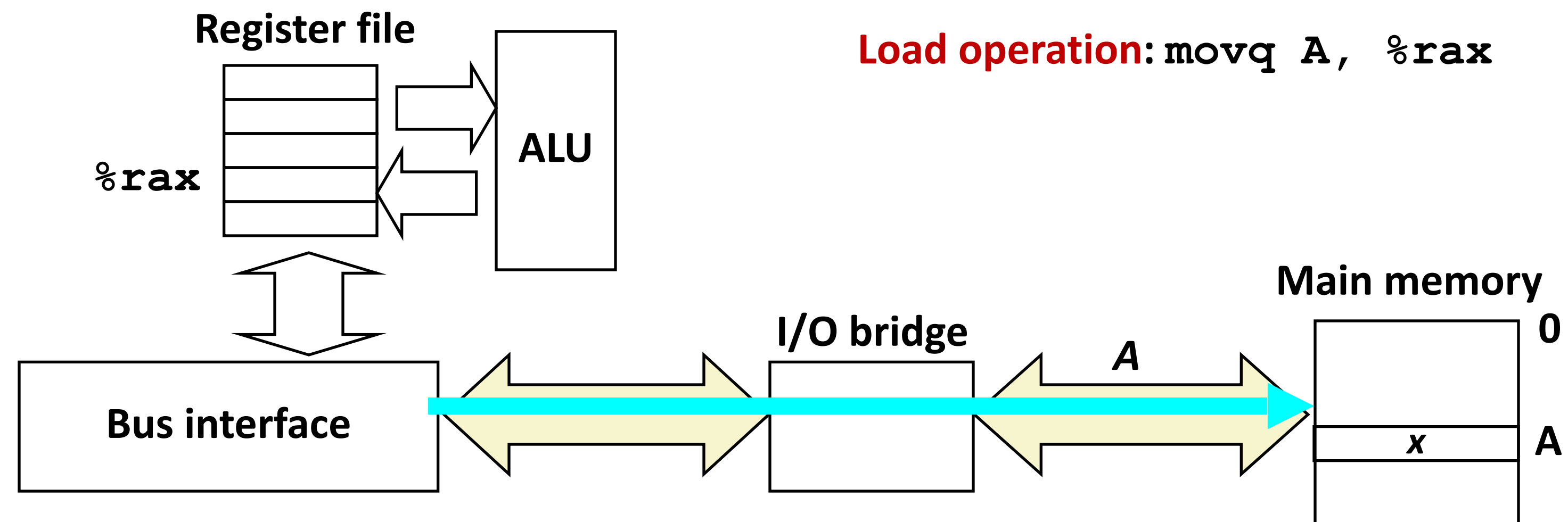


Bus Structure Connecting CPU and Memory

A **bus** is a collection of parallel wires that carry address, data, and control signals. Buses are typically shared by multiple devices.

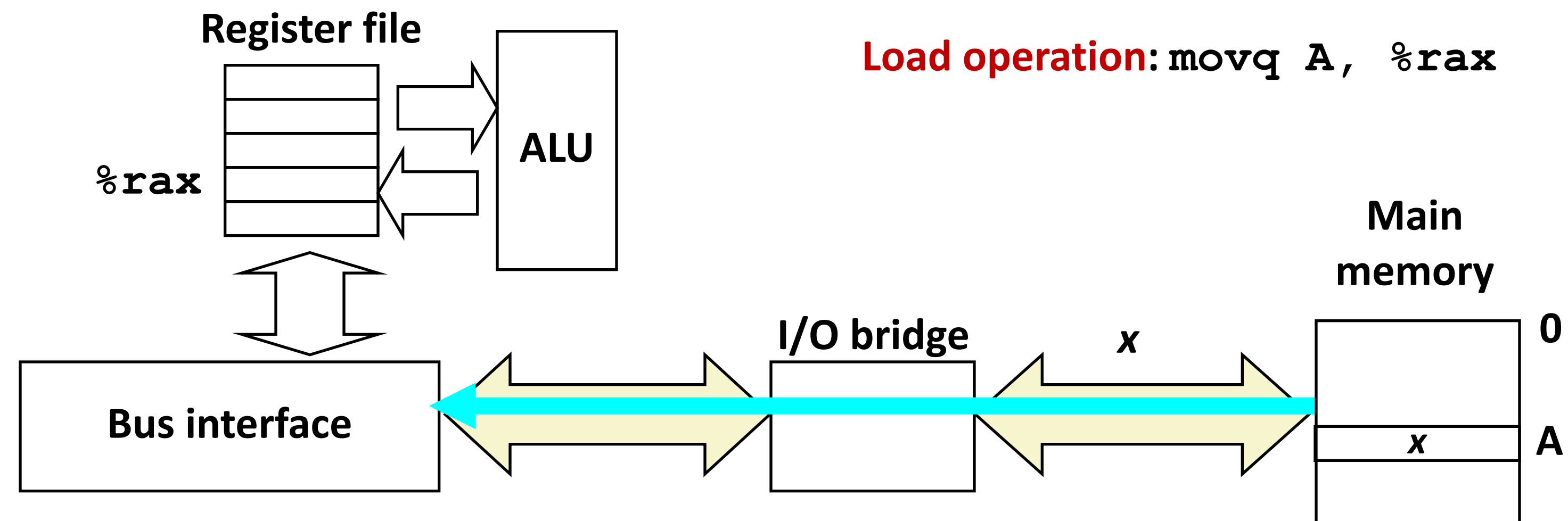


Memory Read Transaction (1)



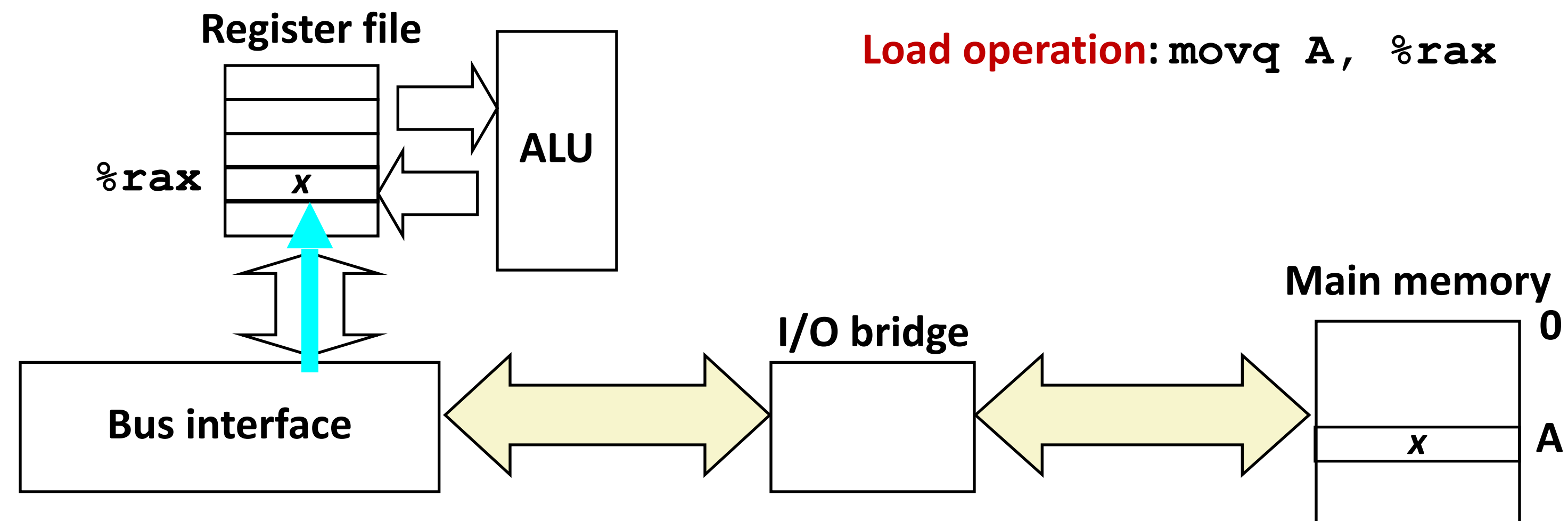
CPU places address `A` on the memory bus.

Memory Read Transaction (2)



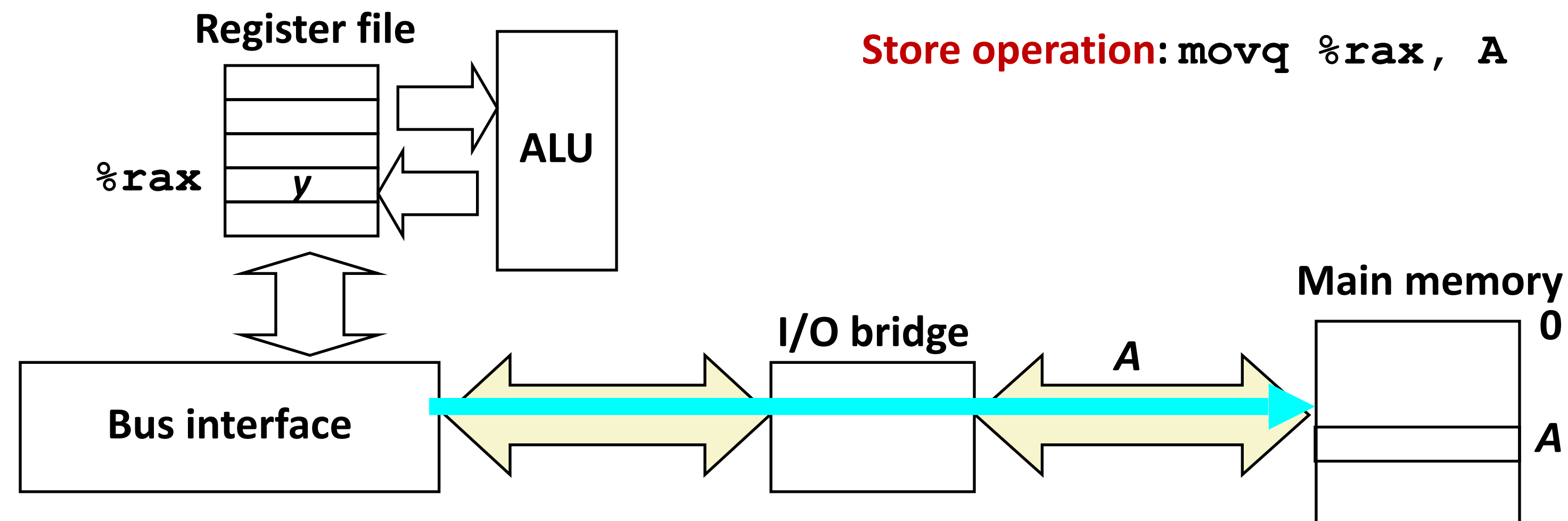
Main memory reads `A` from the memory bus, retrieves word `x`, and places it on the bus.

Memory Read Transaction (3)



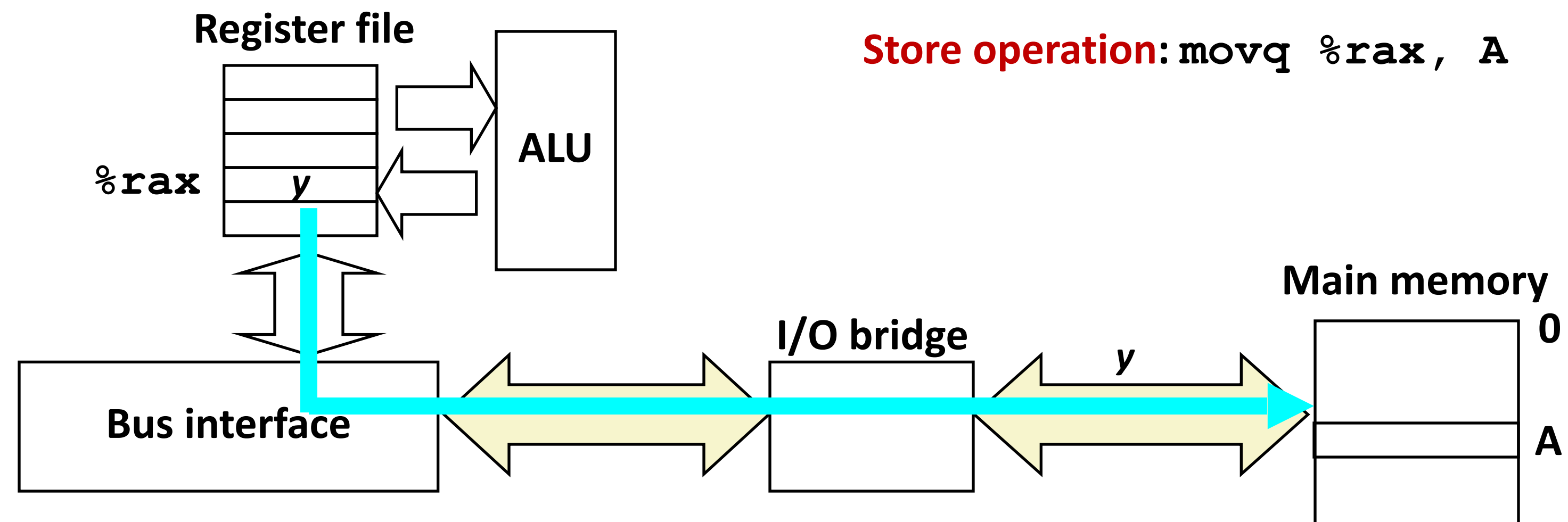
CPU reads word `x` from the bus and copies it into register `%rax`.

Memory Write Transaction (1)



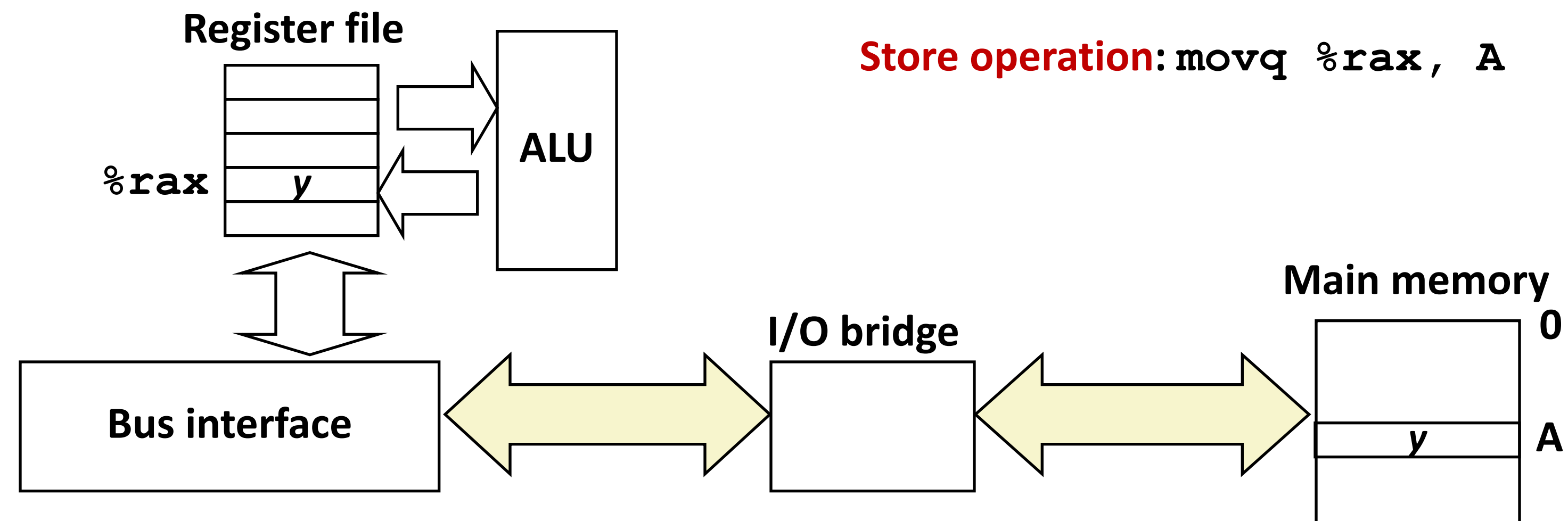
CPU places address *A* on bus. Main memory reads it and waits for the corresponding data word to arrive.

Memory Write Transaction (2)



CPU places data word `y` on the bus.

Memory Write Transaction (3)



Main memory reads data word `y` from the bus and stores it at address `A`.

Basics of Processors

Q: *How does a processor execute machine code?*

- Types of ISA commands to manipulate register contents:
 - **Memory access: load** (copy bytes from a DRAM address to register); **store** (reverse); put constant
 - **Arithmetic & logic** on data items in registers: add/multiply/etc.; bitwise ops; compare, etc.; handled by ALU
 - **Control flow** (branch, call, etc.); handled by CU
- **Caches:** Small local memory to buffer instructions/data

80483b5:	89 e5	mov	%esp,%ebp
80483b7:	83 e4 f0	and	\$0xfffffffff0,%esp
80483ba:	83 ec 20	sub	\$0x20,%esp
80483bd:	c7 44 24 1c 00 00 00	movl	\$0x0,0x1c(%esp)
80483c4:	00		
80483c5:	eb 11	jmp	80483d8 <main+0x24>
80483c7:	c7 04 24 b0 84 04 08	movl	\$0x80484b0, (%esp)
80483ce:	e8 1d ff ff ff	call	80482f0 <puts@plt>
80483d3:	83 44 24 1c 01	addl	\$0x1,0x1c(%esp)
80483d8:	83 7c 24 1c 09	cmpl	\$0x9,0x1c(%esp)
80483dd:	7e e8	jle	80483c7 <main+0x13>
80483df:	b8 00 00 00 00	mov	\$0x0,%eax
80483e4:	c9	leave	
80483e5:	c3	ret	
80483e6:	90	nop	
80483e7:	90	nop	
80483e8:	90	nop	
80483e9:	90	nop	

Example

Q: *What does this program do when run with 'python'?
(Assume tmp.csv is in current working directory)*

tmp.py

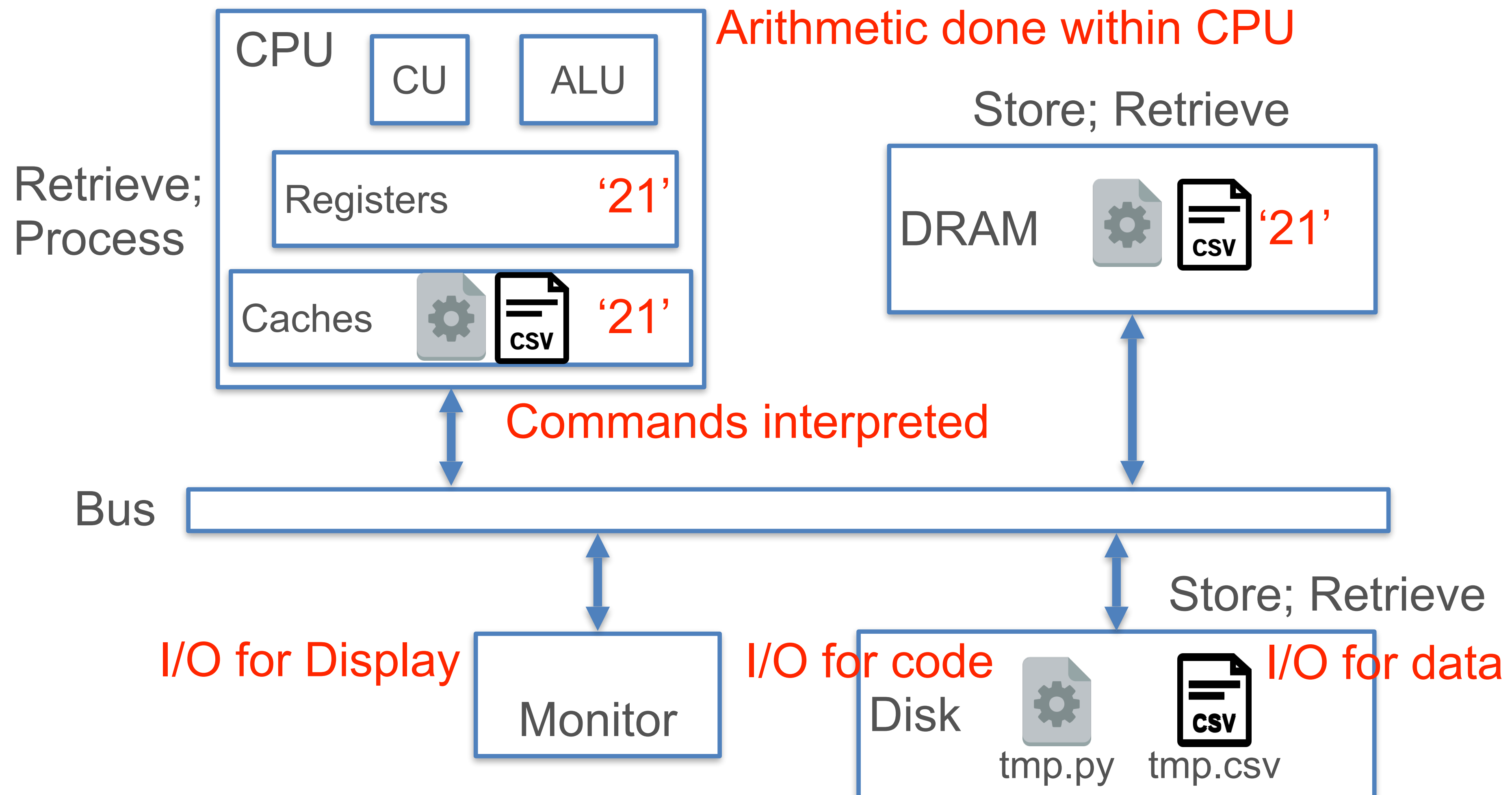
```
import pandas as p
m = p.read_csv('tmp.csv',header=None)
s = m.sum().sum()
print(s)
```

tmp.csv

```
1,2,3
4,5,6
```

Example

Rough sequence of events when program is executed



copyij v.s copyji: copy a 2048 X 2048 integer array

```
void copyij(long int src[2048][2048], long int dst[2048][2048])
{
    long int i,j;
    for (i = 0; i < 2048; i++)
        for (j = 0; j < 2048; j++)
            dst[i][j] = src[i][j];
}
```

4.3 milliseconds

```
void copyji(long int src[2048][2048], long int dst[2048][2048])
{
    long int i,j;
    for (j = 0; j < 2048; j++)
        for (i = 0; i < 2048; i++)
            dst[i][j] = src[i][j];
}
```

81.8 milliseconds